Suicide Text Detection Using NLP Models

Yeji Kim, Anusha Umashankar, Fardin Haiz, Ritu Patel

1. Introduction

The following project aimed to develop an NLP-based platform to analyze whether text messages sent by individuals can be classified as suicidal or non-suicidal. In a time when the internet is such an integral part of daily life, many who are struggling with suicidal thoughts may show glimpses of that online. Current studies suggest that those who are suicidal express their feelings to some degree over the internet before their suicide (Balt, Merelle, Robinson, 2023). The rise in online clues prior to a person's suicide led to the development of this project: a text classifier tool that analyzes user-input text and gives a classification as suicidal or non suicidal and a confidence score. The tool uses multiple NLP models, like a classical Logistic Regression, LSTM with attention, and BERT. The output gives a comparison of these three approaches. A Streamlit application was created to make an effective platform to present these models.

This report is structured into six main sections. The purpose of the introduction is to outline the objectives and motivations behind this project. It serves as a guide to the rest of the report. The second section is the description of the dataset. This is to provide the reader with an understanding of the data used in the project and why it is the correct choice for this project. The next section is the NLP models. This section delves deep into the actual models that are used in the study. This gives insight into the nature of each model and why that model was a good choice for this study. The fourth section is the Experimental Setup. This section illustrates how each NLP model was used in this study, specifically, and the development of the application. It gives insight into the specific challenges and choices made for this dataset and project. The Results section shows the outputs created by each model and different visualizations and metrics to analyze the accuracy of each model. Lastly, the Application section shows the process of the app development and how each of the models was being presented.

2. Dataset

The data used for this project is the “Suicide and Depression Detection” dataset from Kaggle. The dataset takes posts from Reddit users in the “Suicide Watch” and “Depression” subreddits. These are forums where people who experience suicidal thoughts and depression can share their thoughts. The texts taken from these posts are labelled “Suicidal” in the dataset. The other texts in the dataset are from the “Teenagers” subreddit. This is a forum where teenagers can express

their thoughts. The texts from this forum are labelled as “Non-suicidal”. This dataset has 3 columns: number, text, and class. The number is the ID of the row, the text is the exact text from each subreddit, and the class indicates if the text is determined as “suicidal” or “non-suicidal”. When developing this project, this dataset was perfect for developing a suicidal text classifier.

3. NLP model

3.1 Classical TF-IDF + Logistic Regression

The classical NLP model converts raw text into weighted numerical features using Term Frequency–Inverse Document Frequency (TF-IDF), followed by a Logistic Regression classifier for binary suicide detection. This baseline approach establishes a strong point of comparison for the deep learning and transformer models used later in this study.

3.1.1 TF-IDF Vectorization

TF-IDF assigns importance to each term based on its frequency in a document and rarity across the corpus. It is defined as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log\left(\frac{N}{1 + \text{DF}(t)}\right)$$

Where:
- TF(t, d) = frequency of term t in document d
- N = total number of documents
- DF(t) = number of documents containing term t

We use up to 100,000 features with 1–3 gram sequences. Including higher-order n-grams enables the model to detect common multi-word expressions of self-harm intent such as “want to die” and “not worth living.” Sublinear term weighting reduces the dominance of very frequently used words.

3.1.2 Logistic Regression Classifier

Logistic Regression predicts the probability that a post is suicidal using:

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b) \quad \Rightarrow \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Where:
• x = TF-IDF feature vector
• w = learned coefficients
• σ = sigmoid function mapping to [0,1]

This classifier was selected due to its scalability, strong performance on sparse high-dimensional text, and interpretability. Coefficients provide insight into the strength and direction of each term's association with suicide risk.

Cross-Entropy Loss (Binary Logistic Regression):

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

This loss penalizes confident but incorrect predictions and encourages strong separation between suicidal and non-suicidal texts.

3.1.3 Explainability and Feature Interpretation

The learned coefficients show that words such as "suicide", "die", "kill", and "overdose" are highly indicative of suicidal posts, while terms such as "teenager", "meme", and "minecraft" align with non-suicidal content. These patterns validate that the model captures meaningful linguistic signals linked to emotional distress versus casual conversation.

To further analyze model decisions, Local Interpretable Model-Agnostic Explanations (LIME) was applied. LIME highlights specific tokens in each input that strongly influence the classification outcome, confirming that the model recognizes both explicit self-harm language and subtle help-seeking phrases.

This model therefore provides both strong predictive performance and transparent interpretability, which is essential in mental-health screening contexts.

3.2 LSTM

Recurrent neural networks (RNNs) are well-suited for modeling sequential data, but suffer from vanishing gradients and limited long-term memory. To address this, adopting the Long Short-Term Memory (LSTM) architecture, which explicitly controls information flow through gating mechanisms.Further, extending this structure with a Bidirectional LSTM incorporates contextual information from past and future words. Finally, an Attention mechanism is introduced to enable the model to identify and emphasize emotionally relevant tokens, increasing both performance and interpretability.

3.2.1 Recurrent Neural Networks (RNN)

A Recurrent Neural Network processes a sequence one timestep at a time. Given an input sequence

$$\mathbf{x}(1), \mathbf{x}(2), \ldots, \mathbf{x}(T),$$

the RNN updates its hidden state using

$$\mathbf{h}(t) = f\left(W^{xh}\mathbf{x}(t) + W^{hh}\mathbf{h}(t-1) + \mathbf{b}^{h}\right),$$

and produces an output

$$\mathbf{y}(t) = g\left(W^{hy}\mathbf{h}(t) + \mathbf{b}^{y}\right).$$

While conceptually simple, the RNN struggles to remember long-range dependencies due to vanishing or exploding gradients. Posts expressing suicidal intent often rely on nuanced, long-distance language patterns (e.g., "I feel worthless… I do not want to live anymore"). RNNs alone cannot capture such dependencies, motivating the use of LSTMs.

3.2.2 Long Short-Term Memory (LSTM)

LSTMs introduce a memory cell and gating mechanisms that regulate how information is stored, updated, and forgotten over time. This allows the model to maintain relevant emotional or contextual information over longer spans of text.

The LSTM gates are defined as follows:

Feedback gate: $\mathbf{f}(t) = \sigma\left(W^{xf}\mathbf{x}(t) + W^{hf}\mathbf{h}(t-1) + \mathbf{b}^{f}\right)$

Input gate: $\mathbf{i}(t) = \sigma\left(W^{xi}\mathbf{x}(t) + W^{hi}\mathbf{h}(t-1) + \mathbf{b}^{i}\right)$

Candidate cell update:

$$\tilde{\mathbf{c}}(t) = \tanh\left(W^{xc}\mathbf{x}(t) + W^{hc}\mathbf{h}(t-1) + \mathbf{b}^{c}\right)$$

Cell state update:

$$\mathbf{c}(t) = \mathbf{f}(t) \odot \mathbf{c}(t-1) + \mathbf{i}(t) \odot \tilde{\mathbf{c}}(t)$$

Output gate: $\mathbf{o}(t) = \sigma\left(W^{xo}\mathbf{x}(t) + W^{ho}\mathbf{h}(t-1) + \mathbf{b}^{o}\right)$

Hidden state: $\mathbf{h}(t) = \mathbf{o}(t) \odot \tanh(\mathbf{c}(t)).$

Each gate plays a specific role:

- The **feedback gate** selects which information from the previous memory cell should be retained or discarded.
- The **input gate** determines what new information to add.
- The **candidate state** proposes new information to integrate.
- The **cell state** serves as the main memory pathway through time.
- The **output gate** decides what information contributes to the hidden state.

These mechanisms allow the LSTM to model complex emotional progression within text, such as sustained descriptions of hopelessness or self-harm ideation.

3.2.3 Bidirectional LSTM

Suicidal ideation often emerges from contextual cues that appear across the entire sentence, not just preceding tokens. For example, the meaning of ``I am fine" changes drastically when followed by ``but I cannot keep pretending anymore". To capture this bidirectional context, I used a Bidirectional LSTM (BiLSTM).

Two LSTMs process the sequence:

$$\overrightarrow{\mathbf{h}}(t) = \mathrm{LSTM}_{\rightarrow}(\mathbf{x}(t)), \qquad \overleftarrow{\mathbf{h}}(t) = \mathrm{LSTM}_{\leftarrow}(\mathbf{x}(t)).$$

The combined hidden state is:

$$\mathbf{h}(t) = \begin{bmatrix} \overrightarrow{\mathbf{h}}(t) \\ \overleftarrow{\mathbf{h}}(t) \end{bmatrix}.$$

This representation accounts for both past and future dependencies, which is crucial for detecting linguistic cues associated with self-harm or emotional distress.

3.2.4 Attention Mechanism

While LSTMs generate contextual representations, not all words contribute equally to determining whether a post indicates suicidal intent. The Attention mechanism assigns a learned weight to each token.

Score computation: $\mathbf{u}(t) = \tanh\left(W^a \mathbf{h}(t) + \mathbf{b}^a\right)$

Attention weights: $$\alpha(t) = \frac{\exp\left(\mathbf{u}(t)^\top \mathbf{u}^a\right)}{\sum_{\tau=1}^{T} \exp\left(\mathbf{u}(\tau)^\top \mathbf{u}^a\right)}$$

Sentence representation: $$\mathbf{v} = \sum_{t=1}^{T} \alpha(t)\mathbf{h}(t)$$

Interpretation. Attention highlights emotionally meaningful words such as:

- ``worthless"
- ``suicidal"
- ``I want to disappear"
- ``I cannot keep going"

This intrinsic interpretability is essential for mental health applications, where transparency and accountability are required.

3.2.5 Classification Layer

The Attention output is fed into a final dense layer:

$$\hat{\mathbf{y}} = \text{softmax}\left(W^s \mathbf{v} + \mathbf{b}^s\right)$$

where $\hat{\mathbf{y}}$ is the predicted probability distribution over the classes (suicide, depression, or other).

3.2.6 Explainability Techniques

To ensure transparency in model decisions, we incorporate post-hoc interpretability methods.

3.2.6.1 Integrated Gradients (IG)

IG attributes importance to each input feature by computing the integral of gradients from a baseline to the actual input:

$$IG_i(\mathbf{x}) = (x_i - x_i') \int_{\alpha=0}^{1} \frac{\partial F\left(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}')\right)}{\partial x_i}\, d\alpha.$$

Purpose: Measures contribution of each token to the final prediction.

3.2.6.2 SHAP Values

SHAP values quantify each token's contribution based on cooperative game theory:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} \left[f(S \cup \{i\}) - f(S) \right].$$

Purpose: Provides global and local explanations for the model's behavior.

3.3 Transformer Based Model (BERT)

BERT (Bidirectional Encoder Representations from Transformer) is a transformer-based language model trained on large English corpora. Unlike RNN, BERT reads text bidirectionally which enables it to understand context more accurately. This makes BERT well-suited for suicide detection where meaning often depends on subtle wording, emotional tone, and context.
The BERT model consists of multiple transformer encoder layers, each containing multi-head self attention mechanisms and feed forward networks. For this project, I used the bert-base-uncased variant, which contains 12 encoder layers, 768 dimensional hidden states, 12 attention heads and approximately 110 million parameters. The model was pretrained on the Toronto Books Corpus and English Wikipedia providing it with broader linguistic knowledge before fine-tuning on our specific task.

3.3.1. Algorithm and Equations

To adapt BERT for our specific binary classification task, I implemented the following architecture.

3.3.2. Input Representation

Each input sequence begins with a special [CLS] token followed by the tokenized text and ends with a [SEP] token. The input representation combines token embeddings, segment embeddings and positional embeddings

$$h_0 = E_{token} + E_{position} + E_{segment}$$

3.3.3. Contextual Encoding

The input *h*_0 passes through 12 Transformer blocks. Each block updates the contextual representation of the words using self-attention.

$$h_l = \text{TransformerBlock}(h_{l-1})$$

3.3.4. Classification Head

To repurpose BERT for suicide detection, I extracted the final hidden state of the [CLS] token (h_{CLS}^{L}) . I passed this vector through a custom linear layer and applied a dropout of 0.1 to prevent overfitting.

$$z = W h_{CLS}^{L} + b$$

W reduces the dimension from 768 to 2 (Suicide vs Non-Suicide).

3.3.5. Loss Function

During the training, I optimized the model using Cross-Entropy Loss, which penalized the model for confident but wrong prediction.

$$L = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{C} y_{ik} \log p(y_k \mid x_i)$$

4. Experimental setup

4.1 CLASSICAL

The dataset was preprocessed through cleaning, lemmatization, and stopword removal, then split into stratified train, validation, and test sets. Text was transformed using TF-IDF with extended n-grams to capture contextual patterns. Logistic Regression with class-balanced weighting served as the baseline classifier. Hyperparameter tuning with GridSearchCV ensured optimal performance. Model evaluation was conducted using accuracy, macro-F1, ROC-AUC, and confusion matrices to assess both overall performance and error distribution.

Table: Experimental Setup

Component	**Method**
Train/Val/Test Split	80/10/10 stratified
Text cleaning	Lowercasing, URL removal, lemmatization, stopwords removal
Class imbalance	`class_weight = 'balanced'`
Hyperparameter tuning	GridSearchCV, scoring = macro F1
Evaluation Metrics	Accuracy, precision, recall, macro F1, ROC-AUC
Frameworks used	Scikit-Learn, NLTK, Pandas, Seaborn, Matplotlib

Hyperparameters searched:

```
params = {
    "C": [0.5, 1, 5, 10],
    "penalty": ["l2"],
    "solver": ["lbfgs"]
}
```

Best Parameters:

```
{'C': 10, 'penalty': 'l2', 'solver': 'lbfgs'}
```

4.2 LSTM

To evaluate the proposed BiLSTM-Attention model for suicide risk classification, the dataset was

- Divided into training and validation subsets using an 80/20 split, ensuring that both classes (suicide and non-suicide) were proportionally represented.
- All text samples were lowercased, tokenized, converted into integer sequences using the constructed vocabulary, and padded to a fixed maximum length.
- The corresponding class labels were transformed into numerical encodings using a fitted LabelEncoder.

The model was implemented in PyTorch, where each input sequence was passed through an embedding layer, a bidirectional LSTM encoder, and a trainable attention mechanism that aggregated token-level representations into a context vector for classification. Training was performed using the Adam optimizer with a cross-entropy loss function. During each epoch, the model parameters were updated using mini-batches of size 32. To prevent overfitting, dropout regularization was applied to the LSTM outputs and attention context vector.

Model performance was assessed on the held-out validation set after each epoch. Evaluation metrics included accuracy, macro-precision, macro-recall, and macro F1-score, providing a balanced view of performance across the imbalanced classes. Additionally, the confusion matrix was computed to examine class-specific behavior. For interpretability experiments, attention weights, Integrated Gradients and SHAP values. All experiments were executed on GPU, and the training pipeline automatically logged losses, accuracies, and generated visualizations of the evaluation metrics.

4.2.1 Hyperparameter Search

The primary hyperparameters explored include:

- Learning rate: Searched over $\{1 \times 10^{-4}, 3 \times 10^{-4}, 1 \times 10^{-3}\}$ using the Adam optimizer. The final model used 1×10^{-3} as it provided the best balance between convergence speed and stability.
- Dropout rate: Evaluated dropout values $\{0.2, 0.3, 0.5\}$. A rate of 0.3 helped reduce co-adaptation between LSTM units while maintaining predictive power.

4.2.2 Preventing Overfitting

To avoid Overfitting during training,

- Validation monitoring: After each epoch, model accuracy and loss were computed on a held-out validation set. An increasing gap between training and validation loss was used as an indicator of overfitting.
- Dropout regularization: Applied a dropout rate of 0.3 in the LSTM and fully connected layers to reduce over-reliance on specific neurons.

4.2.2 Preventing Extrapolation Errors

- Vocabulary capping: Rare tokens were replaced with an {<UNK>} token to avoid embedding instability for extremely sparse words.
- Input padding/truncation: All sequences were normalized to a fixed length, ensuring consistent input shapes and avoiding extrapolation outside expected sequence ranges.

4.3. BERT

4.3.1. Data Utilization & Splitting Strategy.

To ensure a robust evaluation, we implemented a stratified splitting strategy to divide the dataset into three distinct subsets while maintaining the original class distribution across all training, validation and test set.

- Training set (80%): used to update the model weights by backpropagation.
- Validation set (10%): used for hyperparameter tuning and early stopping monitoring during training
- Test set (10%): a held out set used for the final performance evaluation reported in the result session.

4.3.2 Model Implementation

We implemented the model using the Hugging Face Transformers library within the PyTorch framework. We initialized the architecture with the pre-trained bert-base-uncased weights and attached a binary classification head (a single linear layer) on top of the [CLS] token output.

- Tokenization: We utilized the BERT tokenizer with a maximum sequence length of 320 tokens, applying truncation and padding to handle variable-length social media posts efficiently.
- Performance Evaluation: We judged the model's performance based on a comprehensive suite of metrics: Accuracy, Precision, Recall, F1-Score, and ROC-AUC. We prioritized the F1-Score during development to ensure a balance between detecting true threats and minimizing false alarms.

4.3.2 Hyper-parameter Search

We utilized the AdamW optimizer with a standard fine-tuning learning rate of 2×10^{-5} and a batch size of 16. While we fixed the optimizer parameters based on established literature for BERT fine-tuning, we conducted a targeted search for the decision threshold.

- Threshold Optimization: Instead of using the default 0.5 probability cutoff, we performed a grid search on the validation set across 81 threshold values running from 0.1 to 0.9 to identify the optimal threshold that maximizes the F1-score. The optimal threshold was determined to be 0.47 which slightly favors recall over precision.

4.3.3 Prevention of Overfitting & Extrapolation

To ensure the model generalizes well to unseen data, we employed several regularization techniques:

- Dropout: We applied a dropout rate of 0.1 to the classification head to prevent neuron co-adaptation.
- Weight Decay: We applied a weight decay of 0.01 to penalize large weights through L2 regularization.
- Early Stopping: We monitored the validation F1 score at the end of every epoch. If the F1 score did not improve for 2 consecutive epochs, training was halted to prevent overfitting to the training data.
- Data Deduplication: During preprocessing, we removed exact duplicates from the dataset. This prevents "data leakage" where the model might otherwise see the exact same post in both the train and test sets, leading to inflated performance estimates.

5. Results

5.1 Classical Model Results

The classical model achieves stable high performance across both validation and test sets, with balanced accuracy and strong generalization.

Table: Classical Model Performance

Dataset	Accuracy	Macro F1
Validation	0.94	0.94
Test	0.94	0.94

5.1.1 Confusion Matrix

The confusion matrix shows both low false-negative and low false-positive rates, demonstrating reliable distinction between suicidal and non-suicidal text.

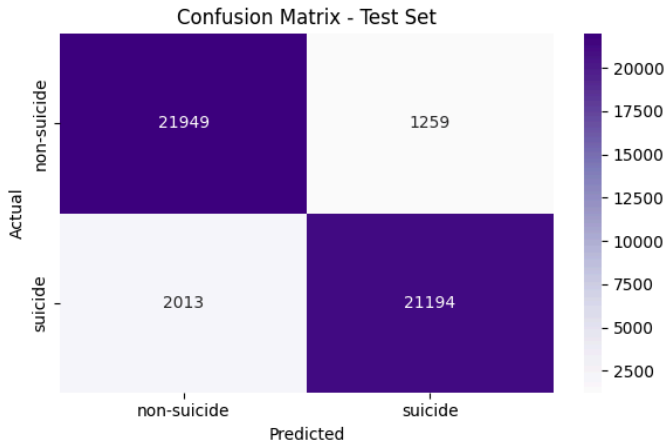


Figure: Confusion Matrix

Test results indicate:

Low false negatives (dangerous misclassification) and Low false positives (unnecessary alarm)
Both are critical for real-world deployment in mental health screening.

5.1.2 ROC-AUC Curve

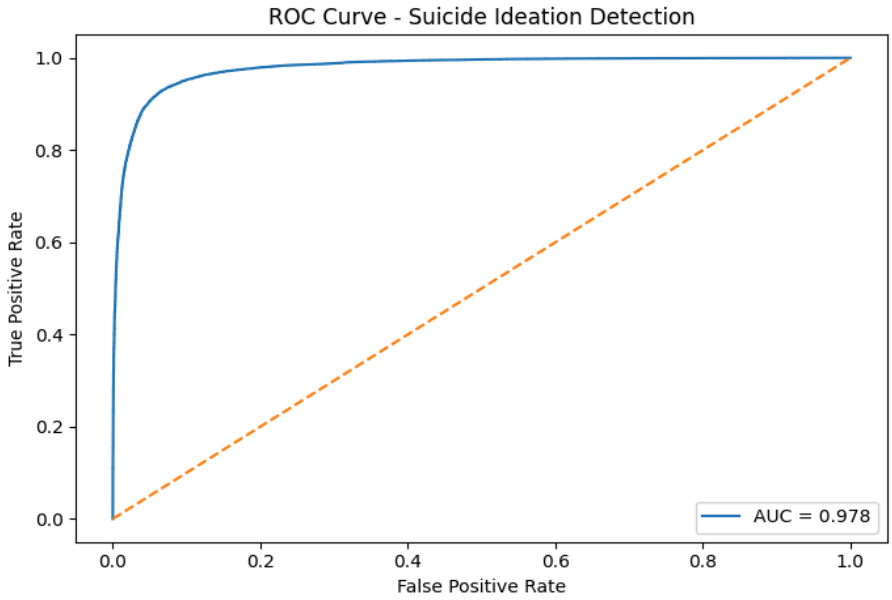


Figure: AUC-ROC Curve

The ROC-AUC score of 0.978 indicates excellent separability between classes and high confidence across varying thresholds.

5.1.3 Feature-Based Interpretability

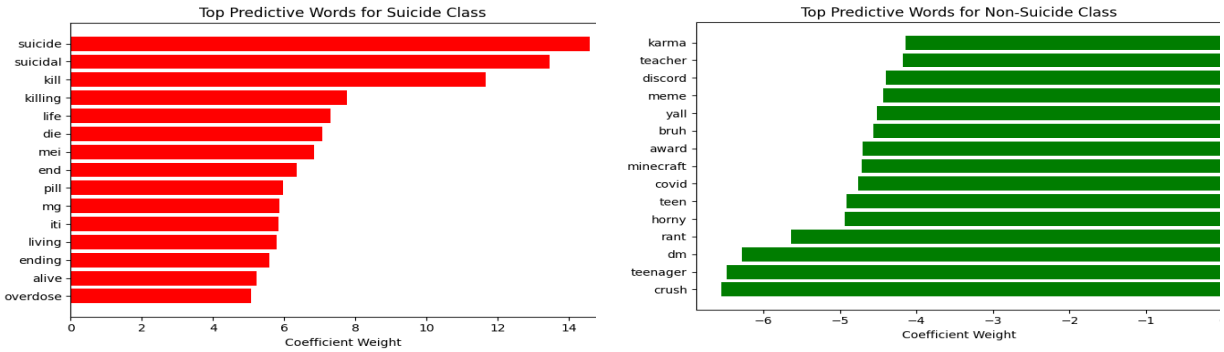


Figure: Topp Predictive Words for Suicide and Non- Suicide Class

Coefficient analysis confirms that suicide-related terms receive strong positive weights, while casual or social interaction terms receive negative weights. This aligns with the known context of the originating subreddits and supports transparency in clinical settings.

5.1.4 Explainability of the Classical Model

To ensure that the prediction outcomes are interpretable and credible for mental-health related decisions, feature-level explainability techniques were applied to the Logistic Regression model.

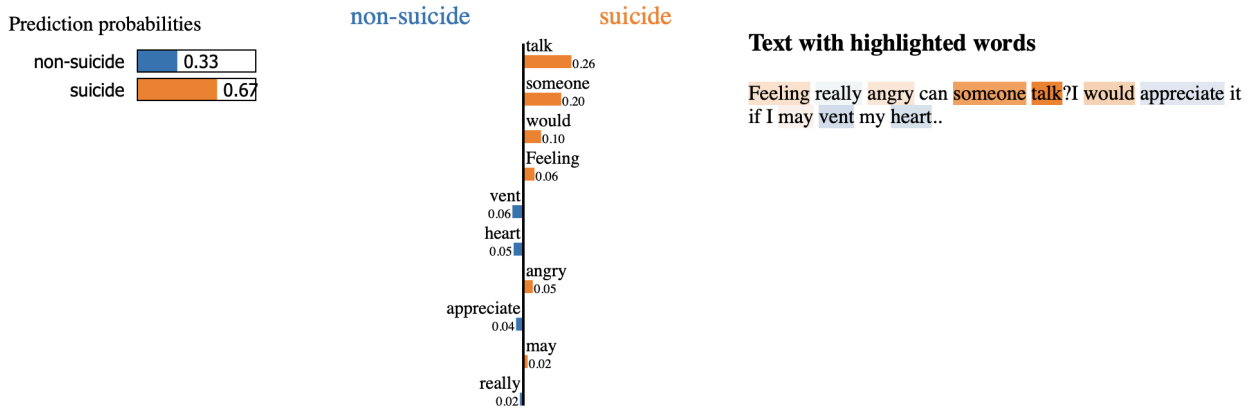


Figure: Explainability via LIME

Coefficient-Based Feature Importance
Since Logistic Regression is a linear classifier, each word feature has a trainable weight. A positive coefficient increases the probability of classifying a post as suicidal, while a negative coefficient reduces it. Let wiw_iwi denote the coefficient for feature xix_ixi. The decision function is:

$$z = \sum_{i=1}^{n} w_i x_i + b \quad \rightarrow \quad \hat{y} = \sigma(z)$$

Terms strongly associated with suicidal ideation included:
• suicide, suicidal, kill, die, overdose, end

Terms strongly associated with non-suicidal posts included:
• teen, school, meme, minecraft, crush

These patterns align with the linguistic context of the source subreddits (r/SuicideWatch vs. r/Teenagers) and verify that the model detects semantically meaningful risk cues rather than random correlations.

LIME-Based Local Explanation

To analyze individual model decisions, Local Interpretable Model-Agnostic Explanations (LIME) was used. LIME generates a sparse surrogate model to approximate the classifier's decision boundary around a specific input sample:

$$g(z') = \arg\min_{g \in G} \Big(\mathcal{L}(f, g, \pi_x) + \Omega(g) \Big)$$

Where:
- f = original Logistic Regression classifier
- g = simplified local linear model
- π_x = proximity measure for locality-based weighting
- Ω(g) = complexity penalty

In an example suicidal-risk post, emotionally negative words such as "talk", "someone", and "feeling" were highlighted as increasing risk, while neutral terms ("vent", "heart") reduced risk slightly. This shows the model is sensitive not only to explicit self-harm vocabulary but also to contextual distress expressions such as help-seeking language.

5.2 LSTM

The performance results of the Bidirectional LSTM with Attention mechanism demonstrate that the model not only performs strongly in classification but also provides interpretable insights into linguistic patterns associated with suicidal ideation.

5.2.1 Classification Metrics

Figure 5.2.1.1 summarizes the quantitative performance of the model on the held-out validation set. The classifier achieves stable and balanced performance across both classes.

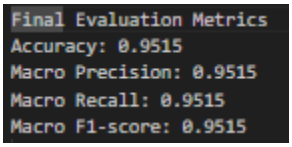


Figure 5.2.1.1 Final evaluation metrics on the validation set.

These results indicate that the model performs equally well in identifying suicidal and non-suicidal posts, with no major bias toward either class.

5.2.2 Confusion Matrix

Figure 5.2.2.1 shows the confusion matrix for the validation set. The model correctly identifies most suicidal posts, misclassifying only a small fraction (979 out of 23,128). Similarly, false positives are limited to 1,273 cases.

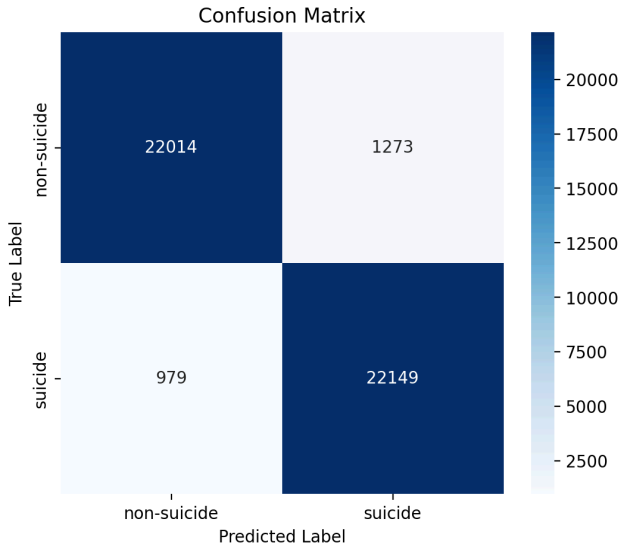


Figure 5.2.2.1 Confusion matrix for the validation set showing strong balanced classification.

The matrix demonstrates that the classifier maintains high sensitivity (recall) toward suicidal content,critical for mental health applications where false negatives must be minimized.

5.2.3 Attention-Based Interpretability

The attention mechanism highlights emotionally salient words. Figure 5.2.3.1 shows an example input where the model assigns high attention weights to tokens such as "*hold*","*apart*", and "*together*", which are strongly indicative of emotional distress.

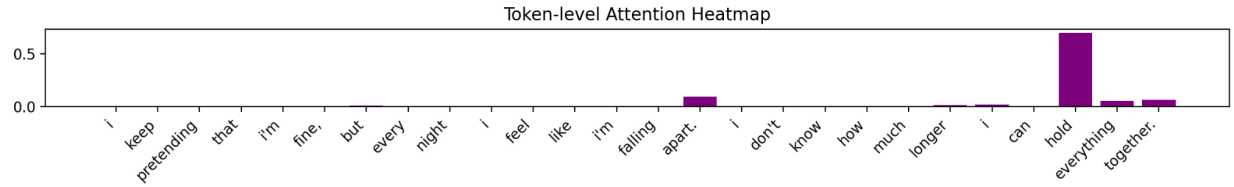


Figure 5.2.3.1 Token-level attention heatmap for a suicidal post. Darker colors indicate higher attention weights.

The distribution of attention weights across all tokens (Figure 5.2.3.2) reveals that while most tokens receive low weights, a subset receives substantially higher values, aligning with key emotional cues.

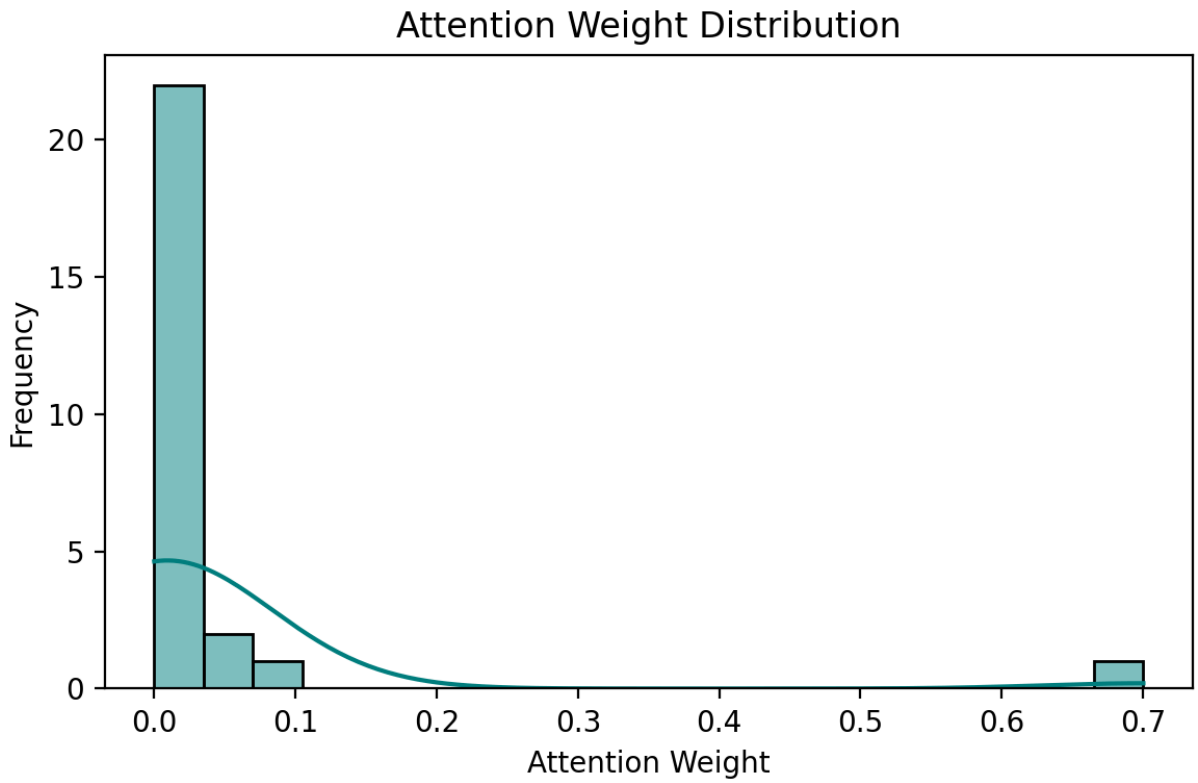


Figure 5.2.3.2 Attention weight distribution showing sparsity and emphasis on emotionally charged words.

5.2.4 Integrated Gradients Analysis

To capture deeper attribution patterns beyond attention, we applied Integrated Gradients (IG). Figure 5.2.4.1 visualizes token-level IG attributions for the same sample. The method highlights similar emotionally significant words, confirming the consistency of the model's interpretive focus.

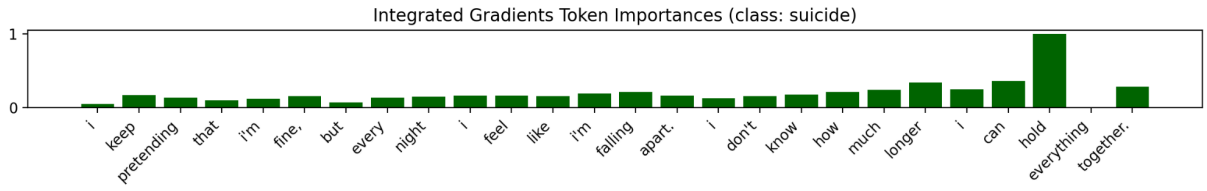


Figure 5.2.4.1 Integrated Gradients token importance for a suicidal input. Higher bars correspond to stronger contributions.

IG provides complementary insight by quantifying how each token contributes to the model's prediction relative to a neutral baseline.

5.2.5 SHAP Value Explanations

To assess global interpretability, SHAP values were computed across multiple samples. Figure 5.2.5.1 shows the SHAP summary plot, illustrating the most influential words in the dataset.

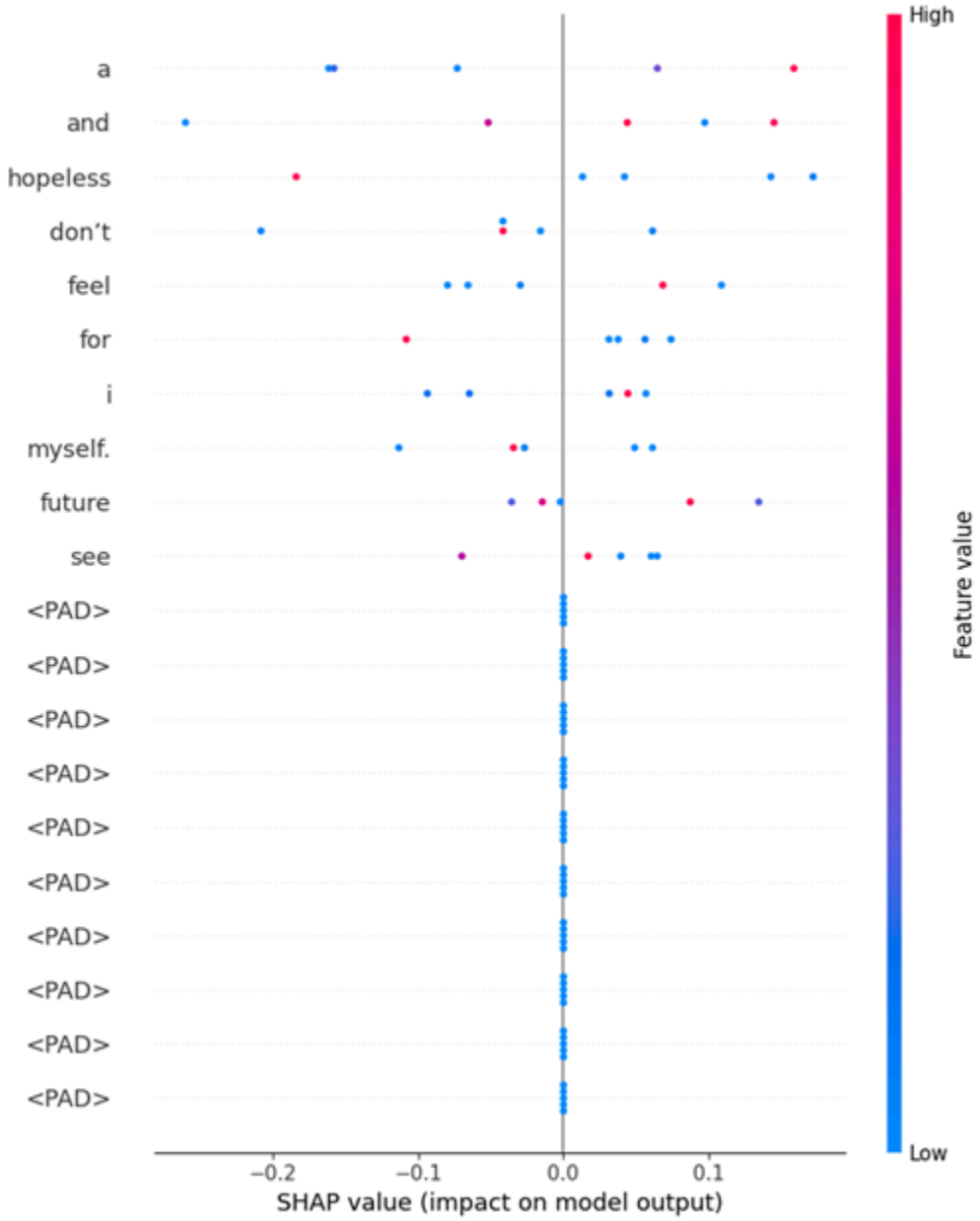


Figure 5.2.5.1 SHAP summary plot showing global feature importance across the dataset.

Words expressing hopelessness, emotional exhaustion, and first-person references (e.g., “myself”, “feel”, “hopeless”) appear among the strongest predictors, aligning with linguistic signals typically associated with suicidal ideation.

5.2.5 Consistency Across Explainability Methods

Across all interpretability techniques - Attention, IG, and SHAP ,the model consistently highlights:

- emotionally negative phrases (e.g., “falling apart”, “don’t know”, “longer hold”)
- self-referential tokens (“I”, “myself”)
- expressions of hopelessness or distress

This convergence increases confidence that the model is learning meaningful semantic patterns rather than spurious correlations.

5.2.6 Summary of Findings

The results demonstrate that:

- The model achieves excellent classification accuracy (95.15%).
- Confusion matrix analysis reveals low rates of false negatives.
- Attention, IG, and SHAP consistently identify key emotional cues.
- Interpretability methods validate that the model aligns with known linguistic markers of suicidal ideation.

Overall, the classifier is both ***high-performing and explainable***, making it suitable for sensitive applications involving mental health text analysis.

5.3 BERT

The fine-tuned BERT model gave impressive results on the held-out test set of 23,123 posts. Table 1 summarizes the key performance metrics.

Table1.

	precision	recall	f1-score	support
0	0.9827	0.9776	0.9801	11595
1	0.9777	0.9828	0.9802	11604
accuracy			0.9802	23199
macro avg	0.9802	0.9802	0.9802	23199
weighted avg	0.9802	0.9802	0.9802	23199

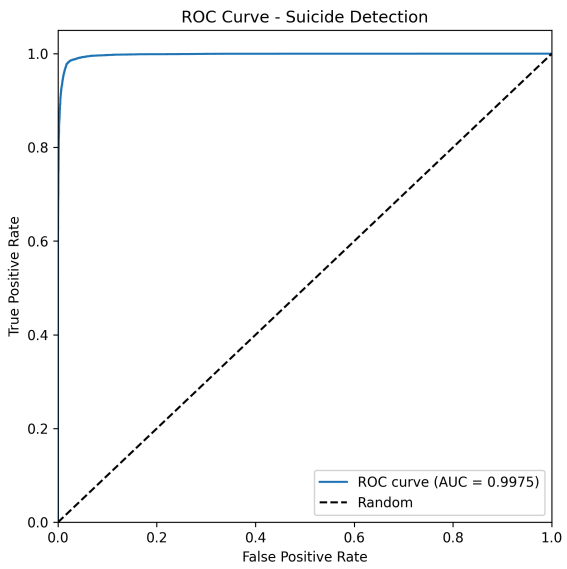
ROC_AUC: 0.9975

The results indicated that the model achieves 98% accuracy while maintaining balanced precision and recall. The AUC scores (0.9975) demonstrate excellent discrimination capability suggesting that the model has learned meaningful patterns that generalize well to unseen data. Also, the balanced precision and recall show that the model avoids bias toward either error type.

5.3.1 ROC Curve

The receiver operating characteristic (ROC) curve plots the true positive rate against the alse positive rate across all possible classification thresholds. The curve hugs the top-left corner of the plot space. This plot confirmed that suicidal and non-suicidal posts from well spearated distributions in the model's learned representation space.

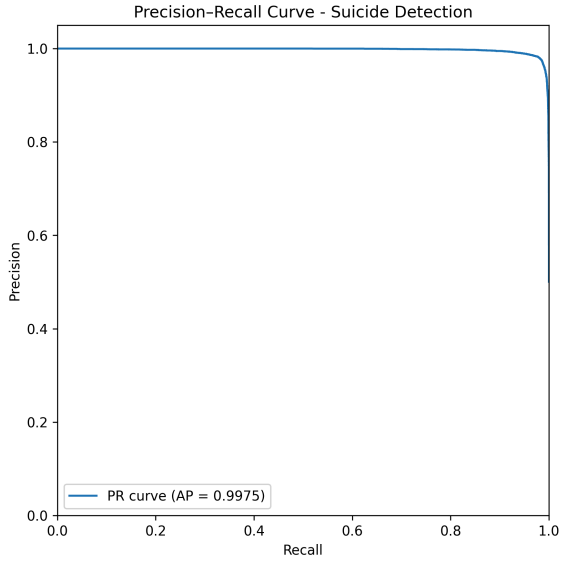
Figure 1. ROC Curve



5.3.2 The Precision-Recall Curve

The precision recall curve provides a detailed view of model performance on the positive class. As shown in the figure, the curve maintains precision above 0.95 across the full range of recall values indicating that it rarely misclassifies non-suicidal posts as suicidal. The Average Precision(AP) scores of 0.9975 illustrates that the classifier ranks suicidal posts ahead of non-suicidal posts with remarkable consistency. The model is able to retrieve the majority of suicidal posts without introducing a substantial number of false alarms. Thus, the PR curve confirms that the classifier is both highly sensitive and precise in distinguishing it from non-suicidal content.

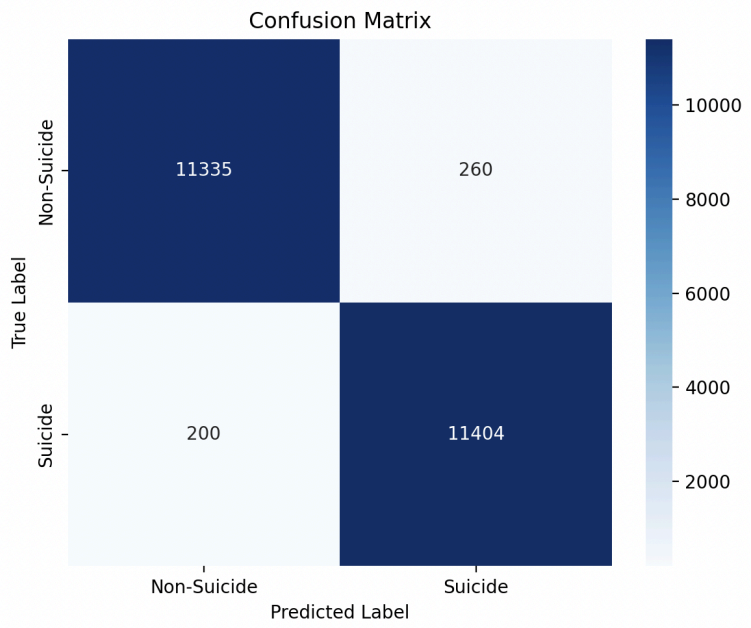
Figure 2. PR Curve



5.3.3. Confusion Matrix

The confusion matrix reveals detailed information of model performance across the two classes. The model correctly identified 11,335 non-suicidal posts and 11,404 suicidal posts indicating strong reliability across both categories. False positive (260 cases) represent posts that the model flagged as suicidal despite being non suicidal. False negative (200 cases) corresponds to suicidal posts that are missed. Since false positive and false negative rates are low and balanced, this result shows that the model is robust across all classes.

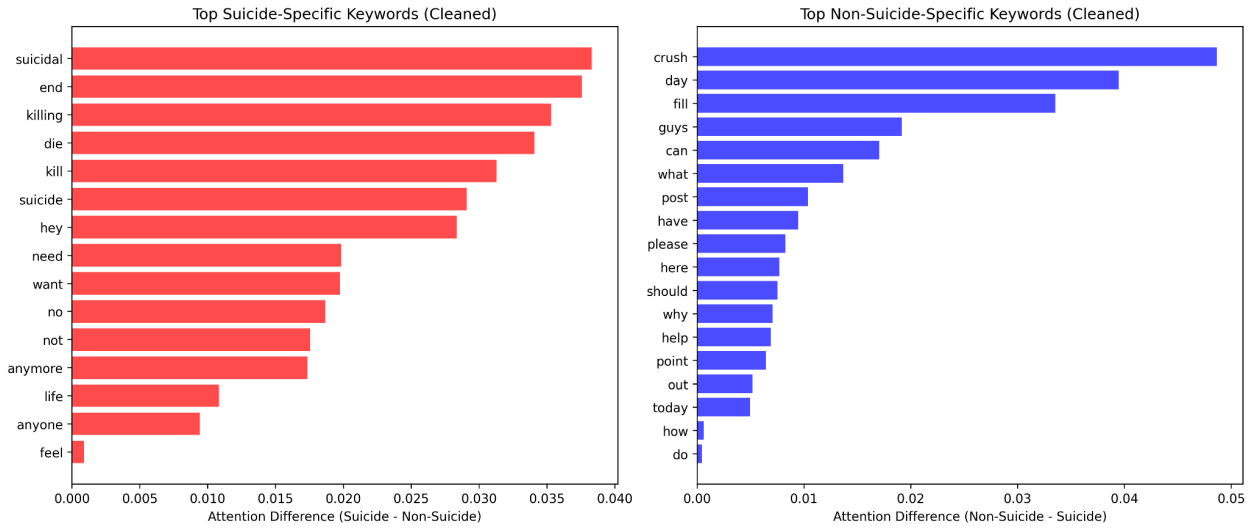
Figure 3. Confusion Matrix



5.3.4. Interpreability

Using the attention aggregation method described in 3.4.1, I successfully identified the linguistic features driving the model's predictions. As illustrated in Figure 4, it presents the top suicidal and non-suicidal keywords after stopword and punctuation filtering. The highest scoring token such as suicidal, end, killing, die, kill, suicide correspond to explicit indicators of suicidal ideation. These words directly describe self-harm or death related expressions. The clear semantic separation between the two keyword groups illustrates that the model's decisions are grounded in meaningful linguistic distinctions.

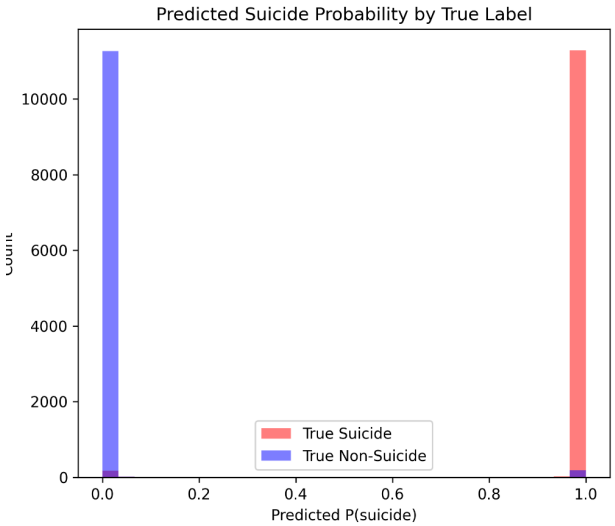
Figure 4. Keyword



5.3.5 Error Analysis

I conducted a comprehensive analysis of the model's failures to understand its limitations. I conducted quantitative and qualitative error analysis. Two figures below show how and why misclassification occurs. The histogram in Figure 5 shows the distribution of predicted suicide probabilities for true suicidal and true non suicidal posts. The separation pattern is consistent with the model's high ROC-AUC score showing the classifier maintains strong discriminative capability across the full probability spectrum.

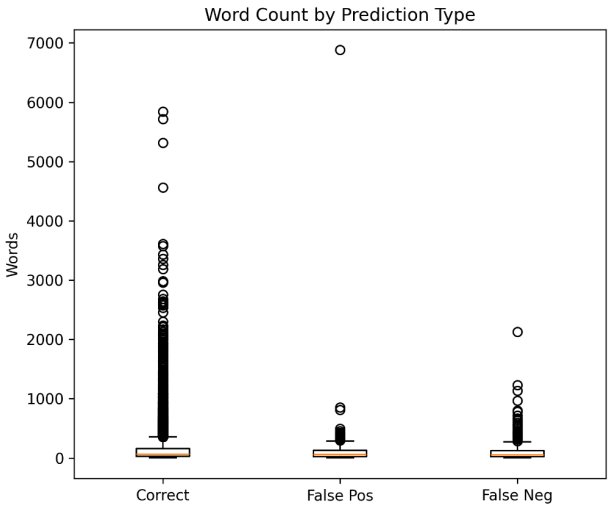
Figure 5. Predicted Suicide Probability Distribution



The figure 6 compares the word count distribution among correct predictions, false positives and false negatives. While the model performs generally well across different lengths, the error analysis reveals a specific vulnerability in short texts. Short inputs containing informal language

or slang lack the contextual redundancy found in longer posts leading to false negatives. In longer posts, the model can recover from slang terms by relying on surrounding context words. Additionally, errors arise from semantic and emotional complexity such as posts using sarcasm, metaphors or indirect expressions of distress without explicit keywords.

Figure 6. Word Count by Prediction Type



6. Application Development

Once all of the models were created, it was important to develop an application that could showcase the results of these separate models. The objective of the application was to make a user-friendly interface that showed the results of each of the separate models and allowed a user to compare the results of these models and make a comprehensive decision if a text could be classified as “suicidal” or “non-suicidal” intentions. This application was developed through a Python package called Streamlit. This is an easy to use package that allows a developer to create an app directly through Python. The layout of the app was then decided to best successfully illustrate the objectives of our project. The application shows a text box that prompts the user to enter text for classification. The user then pressed a button labelled “Run all Models,” which runs each model of the desired text. The output then shows a table that gives the name of the model, the classification, and the confidence. Examples of the application are shown below.

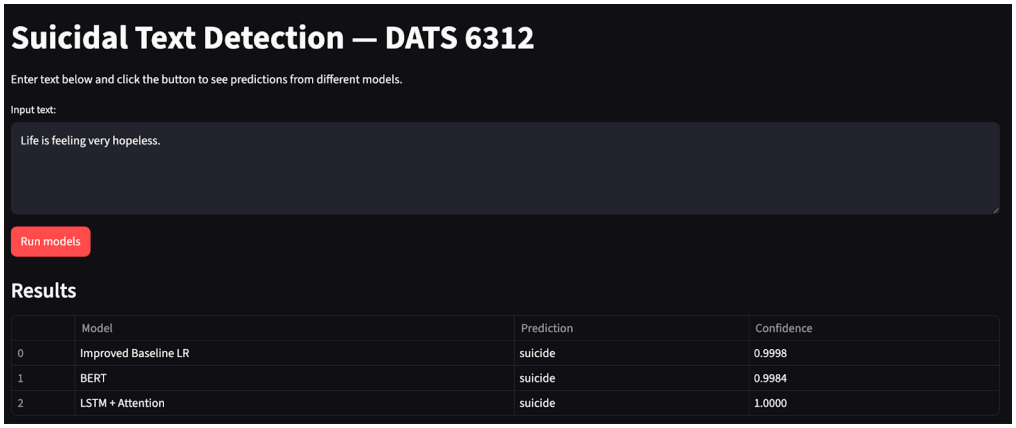


Figure 1- Application being run for the text," Life is feeling very hopeless".

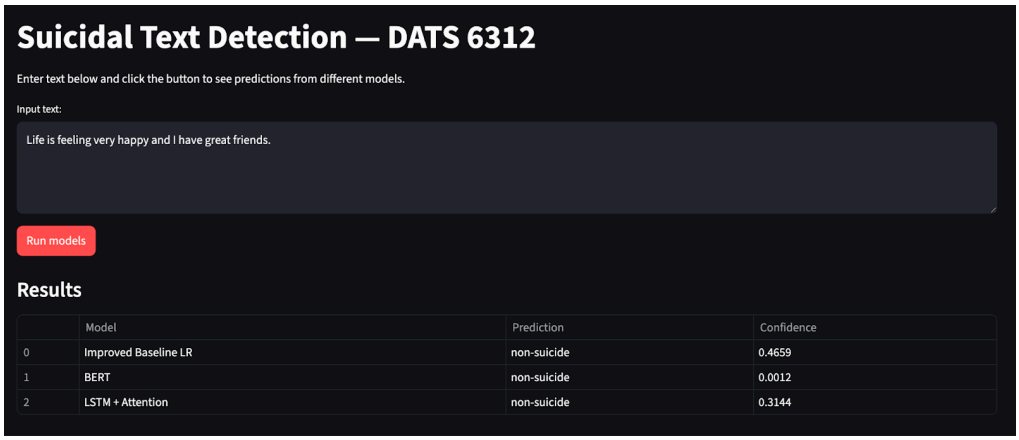


Figure 2- Application being run for the text," Life is feeling very happy, and I have great friends".

7. Conclusion

In conclusion, a successful text-classifying method was developed to determine if texts have suicidal undertones. This was done through the use of multiple Natural Language Processing models of various complexities. By combining the results of all these models, a tool was developed that can comprehensively classify user-input text as suicidal or non-suicidal. Through the process of this project, we learned a lot and also revealed improvements that can be made. The Bidirectional LSTM with Attention demonstrated strong capability in identifying suicidal ideation, outperforming simpler baseline models by capturing long-range emotional dependencies and highlighting the most influential tokens in each prediction.We gained experience with transformer fine-tuning, dataset preparation, interpretability analysis and performance evaluation. The result shows that BERT is highly effective for suicide ideation

detection. However, there are subtle or indirect expressions remaining challenging.Through the development of the application, we learned how to create and visualize code for user-friendly understanding. This portion of the project took different code files and made it one clean tool that can be used by anyone. One potential area of improvement could be to make a tool that can analyze large corpora that have multiple users' messages. For example, if this could become a type of scanner that goes through message boards, it could make it much more efficient to determine who needs help.

8. Reference

- Amir Jafari. NNDesignDeepLearning: Neural network design- deep learning code repository. *https://github.com/NNDesignDeepLearning/NNDesignDeepLearning*.
- Balt, E., Mérelle, S., Robinson, J. et al. Social media use of adolescents who died by suicide: lessons from a psychological autopsy study. Child Adolesc Psychiatry Ment Health 17, 48 (2023). https://doi.org/10.1186/s13034-023-00597-9
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. arXiv preprint arXiv:1810.04805.
- Wolf, T., et al. (2019). *HuggingFace's Transformers: State-of-the-art Natural Language Processing*. arXiv preprint arXiv:1910.03771.
- Vaswani, A., et al. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems.
- https://www.kaggle.com/datasets/nikhileswarkomati/suicide-watch/data
- Komati, N., Vishal, D., Sphoorthi, L., & Fathimabi, S. (2021). *Suicide Ideation Detection in Social Media Forums*. 2021 2nd International Conference on Smart Electronics and Communication (ICOSEC), 1741–1747. https://doi.org/10.1109/ICOSEC51865.2021.9591887